Systematically Tuning Classification Models to Effectively Classify Patients with Heart Disease

Authors: Andrew White, David Faik, Assaf Izhar

Section 1: Introduction

Hyperparameters have a critical role in machine learning performance and the task of tuning them is a large area of study across applications [7, 8]. Optimal hyperparameters depend on the model architecture and the data, making systematic tuning essential for strong generalization–particularly in healthcare classification tasks. The healthcare industry utilizes model classification for tasks such as medical diagnosis, identification of cancerous cells, classification of medical imaging results, and even lung-sound analysis to classify asthma [9, 14]. These diagnostic tasks are inherently high-stakes as model predictions may directly influence life-or-death decisions.

The disease of interest in this project is heart disease, which has been the leading cause of death globally for at least three decades [4, 5, 11]. Using the Personal Key Indicators of Heart Disease dataset from Kaggle, the objective of this project is to tune chosen hyperparameters on perceptron, RandomForest, and multi-layered perceptron (MLP) supervised-learning models to optimally classify heart disease in patients [1]. We then compare the model performance and usability to determine which approach is most suitable for this task.

While previous Kaggle project groups did consider some of the chosen project models (specifically perceptron and RandomForest), they did not report detailed classification performance nor did they systematically vary hyperparameters to achieve optimal tuning [12, 13]. Additionally, although these groups did evaluate multiple models, they had no clear objective once the best model was found.

In this project, we explicitly account for the severity of heart disease by prioritizing the correct identification of affected patients. Hence, our tuned classification models are designed to maximize recall, as false negatives–failing to identify a patient with heart disease could have life-threatening consequences [4-6, 9].

Links for the codebooks for the following sections, along with authorship details, the Kaggle dataset, and the video presentation, are provided in the Appendix (Section 7).

Section 2: Data Information and Processing (Andrew White)

The heart disease data was collected from surveys conducted by the CDC and includes responses from approximately 400,000 adults in the United States and its territories. It has a usability score of 9.41 and there are two versions available: 2020 and 2022. Although the 2022 version was proposed, it does not contain an explicit heart disease target feature for supervised-learning and would require expert annotation to identify cases of heart disease. In contrast, the 2020 dataset contains a binary "HeartDisease" feature, which was used as the target variable in this project (see Fig. 1).

	HeartDisease	BMI	Smoking	AlcoholDrinking	Stroke	PhysicalHealth	MentalHealth	DiffWalking	Sex	AgeCategory	Race	Diabetic	PhysicalActivity	GenHealth	SleepTime	Asthma	KidneyDisease
0	No	16.60	Yes	No	No	3.0	30.0	No	Female	55-59	White	Yes	Yes	Very good	5.0	Yes	No
1	No	20.34	No	No	Yes	0.0	0.0	No	Female	80 or older	White	No	Yes	Very good	7.0	No	No
2	No	26.58	Yes	No	No	20.0	30.0	No	Male	65-69	White	Yes	Yes	Fair	8.0	Yes	No
3	No	24.21	No	No	No	0.0	0.0	No	Female	75-79	White	No	No	Good	6.0	No	No
4	No	23.71	No	No	No	28.0	0.0	Yes	Female	40-44	White	No	Yes	Very good	8.0	No	No

Fig. 1: A snapshot of the data and its features. The target feature is the "HeartDisease" column. It is a mixture of continuous, binary, and multi-label features.

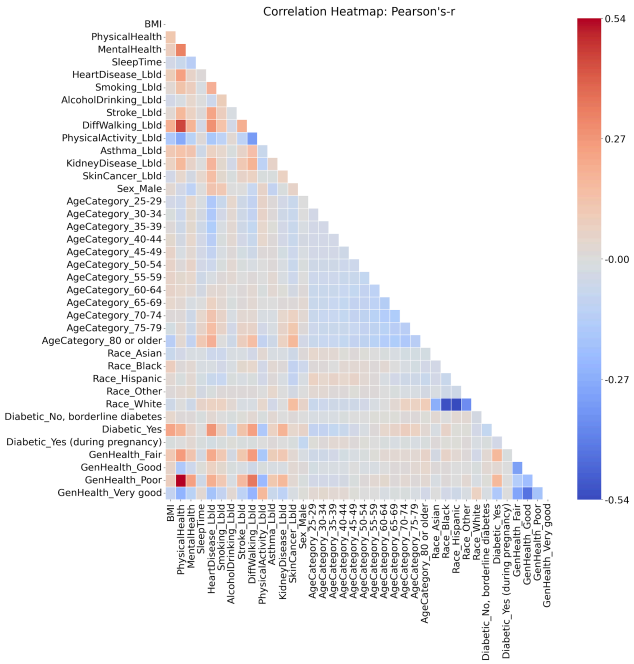


Fig. 2: Correlation heatmap of all encoded and continuous features using Pearson's-r statistic. Note that despite correlations, there is no problematic multicollinearity since strongest r~0.54, which is relatively weak.

The dataset contains 319,795 respondents with a mix of numerical or categorical features, where categorical data are either binary or multi-class. Of the total responses, 27,373 indicate heart

disease and 292,422 do not, corresponding to approximately 9% positive and 91% negative cases. This class imbalance would bias models toward the majority class ("No"). To address this, the dataset was uniformly undersampled at random with respect to the majority class to produce a balanced dataset of 54,746 observations. Previous works utilized SMOTE to rebalance the data, but the dataset was substantial enough that this method was deemed unnecessary for this work [12, 13].

During preprocessing, binary features ("No"/"Yes") entries were mapped to 0 and 1, respectively, and multi-class categorical features were encoded using one-hot encoding via the *get_dummies()* function in the Pandas library. After text-based features were removed, a correlation heatmap was generated using Pearson's-r to examine linear relationships and assess potential multicollinearity (Fig. 2). The results indicate no evidence of problematic multicollinearity, so all features were retained for model training.

The balanced dataset was then prepared for 5-fold cross-validation. Copies of the dataset and "*HeartDisease*" target variable were retained for final model training and testing. Continuous features in the final training and test sets were standardized using *StandardScaler()* function from *scikit-learn*. The dataset was then split into five training and validation folds using the *StratifiedKFold()* function from the *sklearn.model_selection* module to preserve the class balance in each split. For each fold, *StandardScaler()* learned the scaling parameters of the training subset and applied it to both training and validation subsets. This prevents data leakage and ensures that validation data is treated as unseen during model evaluation.

Section 3: Perceptron (David Faik)

The Perceptron is a simple linear binary classifier. Unlike gradient-based methods that compute a loss over the entire dataset, the Perceptron updates weights only when a sample is misclassified, scaling the adjustment by the learning rate. It converges when the data is linearly separable; otherwise, it oscillates around a low-error solution without fully converging.

In this work, the Perceptron was instantiated through the scikit-learn Python library as a single-layer classifier with a linear decision boundary and no hidden activation function. Loss was minimized via the Perceptron update rule, and weight updates were performed using stochastic gradient descent, with the eta0 parameter controlling the step size on each

misclassification. The chosen hyperparameters to vary were learning rate (eta0), regularization type and strength (penalty and alpha), convergence tolerance (tol), and maximum iterations (max_iter). The learning rate determines how aggressively the weight vector is adjusted on each misclassified sample. The penalty and alpha parameters jointly control the type (L1, L2, or ElasticNet) and strength of regularization applied to the learned weights, which constrains model complexity and reduces overfitting. The convergence tolerance sets the threshold below which training terminates early if improvement stalls, and the maximum iterations parameter caps the total number of passes over the training data.

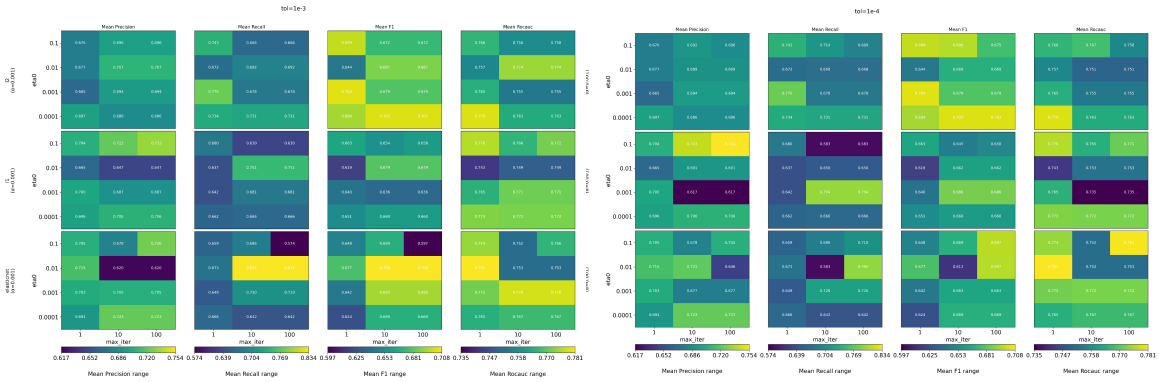


Fig. 3: Heatmap of Perceptron mode model using tol = 1e-3 and tol = 1e-4, precision, recall, f1, and ROC-AUC score metrics from top to bottom. Metrics are color-coded with the highest value in yellow and lowest in dark purple. Values of metrics are mean values taken across 5 - validation folds.

A heatmap (Fig. 3) was generated to visualize mean model performance across the five cross-validation folds for all hyperparameter configurations. In total, the grid search evaluated 168 unique configurations, corresponding to 840 individual model fits under 5-fold cross-validation. Visually, the maximized mean recall corresponds to penalty = ElasticNet, alpha = 0.001, eta0 = 0.010, tol = 0.001, and max_iter = 10 or 100. Since both values of max_iter produced equivalent mean recall, the simpler model with max_iter = 10 was selected in accordance with the principle of preferring lower model complexity when performance is equal.

(A)

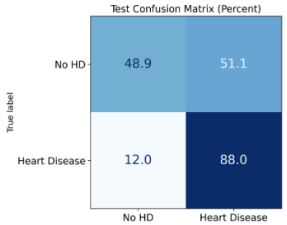
```
inal Model Performance Metrics
verall Accuracy: 0.6846
verall Recall: 0.8803
verall Precision: 0.6327
verall f1: 0.7362
OC-AUC: 0.7850

lassification Report:
              precision    recall  f1-score   support

       No HD       0.80      0.49      0.61      8212
eart Disease       0.63      0.88      0.74      8212

    accuracy                           0.68     16424
   macro avg       0.72      0.68      0.67     16424
weighted avg       0.72      0.68      0.67     16424
```

(B)



(C)

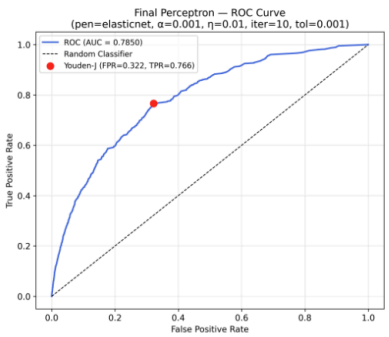


Fig. 4: Performance metrics of Perceptron after final training and evaluation on test data. Hyperparameters were selected from the grid search by maximizing mean recall across the 5 cross-validation folds. Classification report (A) and confusion matrix (B) were generated using the default 0.5 classification threshold. (C) The ROC curve shows strong discriminative performance based on the overall shape of the curve. The red dot marks the threshold maximizing Youden's J on the test set, which falls near 0.322.

Using these hyperparameters, the model was trained on the full training set and evaluated on the held-out test set. A classification report, confusion matrix, and ROC curve were generated and are shown in Fig. 4. The ROC-AUC of 0.785 indicates that the Perceptron has reasonable discriminative ability as a linear classifier on this dataset. At the selected threshold, the model achieved a recall of 0.8803 and a false positive rate of 0.322. Model performance was optimized by selecting hyperparameters which maximized recall across validation folds, consistent with the objective of minimizing false negatives in a heart disease screening context, where failing to identify a positive case carries greater clinical cost than a false alarm.

Section 4: RandomForest (Assaf Izhar)

The second classification algorithm chosen was the Random Forest. Random Forest is a type of bagging (bootstrap aggregation) classification algorithm, where the outputs of multiple decision trees, each trained on a random subset of the dataset and a random subset of features at each split, are combined to produce a result which has a lower variance and is unlikely to overfit the training data.

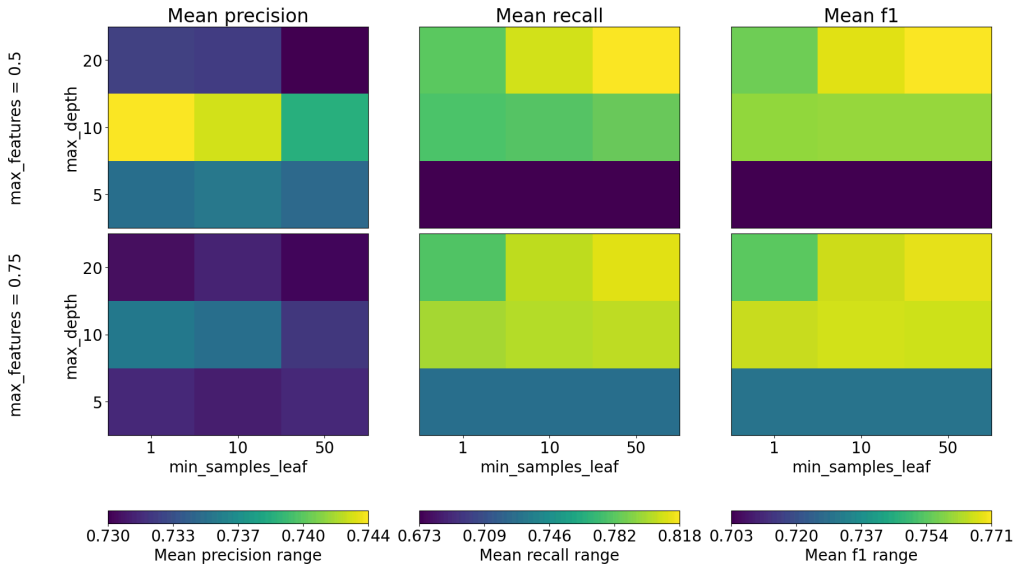


Fig. 5: Heatmap of Random Forest model precision, recall, and f1-score metrics from top to bottom. Metrics are color-coded with the highest value in yellow and lowest in dark purple. Values of metrics are mean values taken across 5 - validation folds.

In this work, the Random Forest was instantiated through the Sci-Kit Learn Python library with a default of n_estimators = 100. The chosen hyperparameters to vary were Maximum Features, Maximum Depth, and Minimum Samples per Leaf. Maximum Features limits the proportion of

random features a given decision tree uses, Maximum Depth limits the depth of any given random decision tree, and Minimum Samples per Leaf restricts the minimum number of samples required at a terminal node, preventing overfitting.

A heatmap (Fig. 5) was generated to determine the mean model performance across folds. Visually, the maximized mean recall corresponds to min_samples_leaf = 50, max_depth = 20, and max_features = 0.5. Fig. 5 also indicates that there is a precision-recall trade-off in the model, where maximum precision is achieved with moderately deep, weakly regularized trees (max_depth = 10, min_samples_leaf = 1), while maximum recall requires deeper trees with stronger leaf-level regularization. This means that in the dataset, capturing more true positive cases requires the model to grow deeper trees and generalize at the leaves rather than memorize specific training examples. Shallow trees (max_depth = 5) consistently underfit across all configurations, indicating insufficient model capacity.

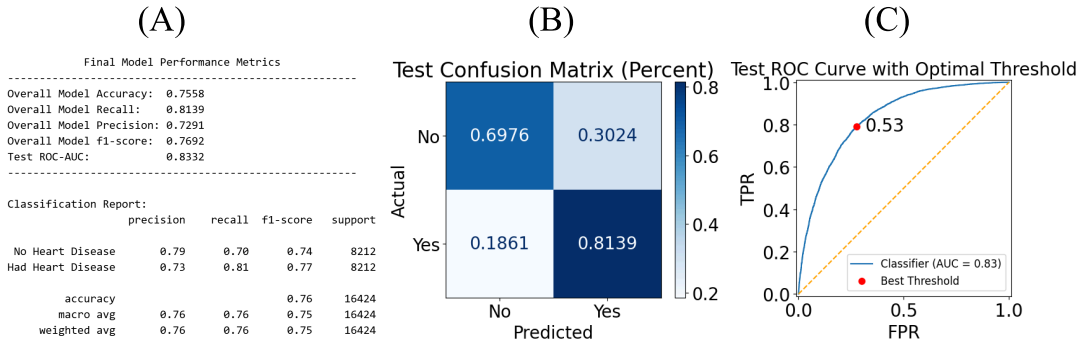


Fig. 6: Performance metrics of Random Forest after final training and evaluation on test data. Hyperparameters were selected from the grid search by maximizing mean recall across the 5 cross-validation folds. Classification report (A) and confusion matrix (B) were generated using the default 0.5 classification threshold. (C) The ROC curve shows strong discriminative performance based on the overall shape of the curve. The red dot marks the threshold maximizing Youden's J on the test set, which falls near 0.50.

Using these hyperparameters, the model was trained and evaluated on the full training and testing set. A classification report, confusion matrix, and ROC curve were generated and indicate that the model performed strongly [Figure 6]. The Model performance was optimized by choosing the model's hyperparameters which produced the highest overall recall, since false negatives carry greater cost than false positives in a heart disease screening context. The ROC Curve shows that the AUC is 0.83, which indicates that the Random Forest is a very strong classifier. At the current threshold value, recall is 0.8139, however increasing the threshold value will decrease the recall while increasing the precision.

Section 5: Multi-Layer Perceptron (Andrew White)

The multilayer perceptron (MLP) was chosen as the final classification algorithm since it is a more complex model which can capture nonlinear relationships. The neural network learns a mapping of inputs to outputs through a sequence of interconnected layers of neurons. Each neuron computes a weighted linear combination of inputs, applies a nonlinear activation function, and then the evaluation is pushed forward to the next layer. By stacking more and more layers, the model can learn more complex relationships. The MLP is trained using backpropagation which adjusts the weights by minimizing a loss function using an optimization scheme (e.g. gradient descent) [5].

In this work, the MLP was constructed using PyTorch libraries and has two hidden layers. The activation function used for outputs was the ReLU function, the loss function was *BCEWithLogitsLoss* which combines the *BCELoss* function with a sigmoid layer for numerical

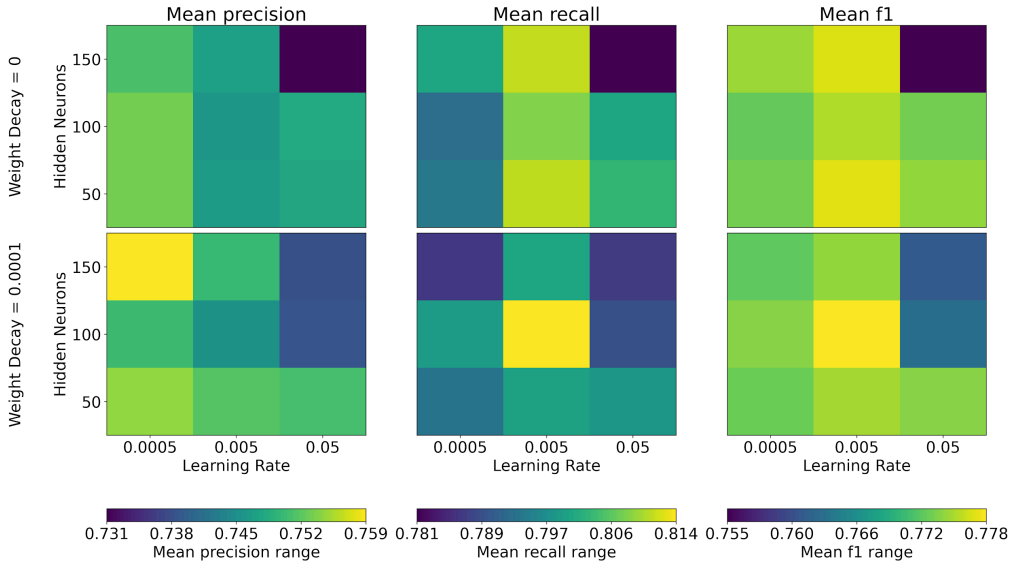


Fig. 8: Heatmap of MLP model precision, recall, and f1-score metrics from top to bottom. Metrics are color-coded with the highest value in yellow and lowest in dark purple. Values of metrics are mean values taken across 5 - validation folds.

stability, and the optimizer was the Adam method [2, 3]. The chosen hyperparameters to vary were weight decay, the number of hidden neurons, and the learning rate. The weight decay applies a penalty to large weights to prevent overfitting, hidden neurons determine the model's capacity to learn new mappings, and the learning rate quantifies the rate of the optimization.

A heat map was generated to determine mean model performance across folds (Fig. 8). Visually, the maximized mean recall corresponds to learning rate η=0.005, weight decay = 0.0001, and

100 hidden neurons. Small learning rate η=0.0005 corresponds to lower recall, precision, and f1-score across folds–this indicates underfitting. The larger learning rate η=0.05 shows worsening mean metrics as well, indicating that overshooting is occurring due to large learning steps. Fig. 8 indicates that the model is insensitive to varying the quantity of hidden neurons and the weight decay values. Thus, MLP model performance was driven by proper optimization.

Using the tuned hyperparameters, the model was trained and evaluated on the full training and test set. A classification report, confusion matrix, and ROC curve was generated and indicates that the model performed very well (Fig. 9). Model performance was optimized by using an

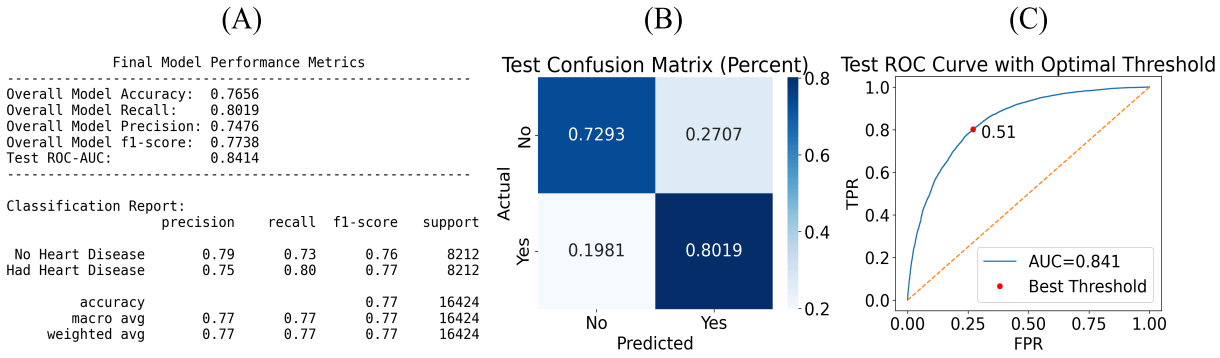


Fig. 9: Performance metrics of MLP after final training and evaluation on test data. Classification report (A) and confusion matrix (B) optimized using median activation threshold value across folds maximizing recall. (C) The ROC curve was generated and shows very good fitting based on shape of curve and a likelihood of false positives. The activation threshold used to generate (A) and (B) in represented by the red dot in (C) and corresponds to the optimal, median Youden's-J across the 5-folds.

activation threshold corresponding to the median Youden's-J statistic taken across validation folds. This statistic is a common technique to find thresholds in activation functions and the median was used since it is less sensitive to outliers compared to the mean [10]. The ROC Curve shows that the AUC (Area-Under-Curve) is 0.841, which indicates the neural network is a strong classifier for true positives. The recall = 0.8019 indicates that the MLP is less-likely to misclassify a true positive as a false negative. In the classifying patients with heart disease, this is a strong recall value to have and means that it is less likely to classify patients with heart disease as not having it, which would be the difference between life and death.

Section 6: Discussion

In our simulations, the perceptron achieved the highest classification recall (Recall = 0.88), compared to the RandomForest (0.81) and MLP (0.80). This is somewhat unexpected, as the perceptron can only model linear relationships, whereas the RandomForest and MLP can capture nonlinear patterns. However, the perceptron exhibited lower accuracy (ACC = 0.68) and precision (precision = 0.63), while the RandomForest and MLP achieved higher and comparable performance (ACC = 0.76, precision = 0.73 and ACC = 0.77, precision = 0.75, respectively).

This indicates that predictions made by the RandomForest and MLP are overall more reliable and less likely to classify patients as false positives; meaning, patients are less likely to be incorrectly classified as having heart disease. Such misclassifications could result in unnecessary medical costs and financial burden. Hence, models that minimize these outcomes should be preferable in practice.

Previous work by the MLFAMM group reported that a Naïve Bayes classifier outperformed their selected models, which included RandomForest and Perceptron architectures [12]. Their data preprocessing involved both SMOTE-based oversampling and NearMiss undersampling. However, their results highlight significant trade-offs: the oversampled model achieved a recall score of 0.33, and while the undersampled model achieved a recall score of 0.98, it achieved a low accuracy of 0.09. These results suggest poor overall performance compared to our models.

A direct comparison of RandomForest models further demonstrates this difference. The *MLFAMM* group reported recall values of 0.26 (SMOTE) and 0.58 (NearMiss), whereas our tuned RandomForest achieved a recall of 0.81. This represents a substantial improvement in classification of true positives and substantial decrease in false negatives. These findings emphasize that effective data preparation and systematic hyperparameter tuning are both critical to model performance.

In terms of implementation, RandomForest is likely more accessible for coders in the healthcare industry. While the MLP can benefit from GPU acceleration and potentially faster training, it requires access to GPU resources and greater familiarity with frameworks such as PyTorch. In this project, all models required similar tuning time (~45 minutes), with final training and testing completed in approximately two minutes on CPU resources.

Therefore, considering both performance and ease of implementation, the RandomForest model is recommended for this classification task.

Section 7: Appendix

Dataset: Personal Key Indicators of Heart Disease (2020 Ver.)

Video Presentation: Video Link

Perceptron Code (David Faik): Perceptron

RandomForest Code (Assaf Izhar): Random Forest

Preprocessing and MLP Code (Andrew White): Preprocessing and MLP

Section 8: References

1. 2020 Version “Personal Key Indicators of Heart Disease. ”, https://www.kaggle.com/datasets/kamilpytlak/personal-key-indicators-of-heart-disease/data
2. Kingma, Diederik P., and Jimmy Ba. "Adam: A method for stochastic optimization." *arXiv preprint arXiv:1412.6980* (2014).
3. “BCEWithLogitsLoss.” *PyTorch Documentation*, PyTorch, https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html. Accessed 27 Apr. 2026.
4. “Mayo Clinic Staff.” *Heart Disease: Symptoms and Causes.* Mayo Clinic, https://www.mayoclinic.org/diseases-conditions/heart-disease/symptoms-causes/syc-20353118. Accessed 26 Apr. 2026.
5. Cleveland Clinic. *Heart Disease.* Cleveland Clinic, https://my.clevelandclinic.org/health/diseases/24129-heart-disease. Accessed 26 Apr. 2026.
6. Safdar, S., Zafar, S., Zafar, N. *et al.* “Machine learning based decision support systems (DSS) for heart disease diagnosis: a review.” *Artif Intell Rev* **50**, 597–623 (2018). https://doi.org/10.1007/s10462-017-9552-8
7. Arnold, Christian et al. “The Role of Hyperparameters in Machine Learning Models and How to Tune Them.” *Political Science Research and Methods* 12.4 (2024): 841–848. Web.
8. Matthias Feurer, and Frank Hutter. “Hyperparameter Optimization.” *Automated Machine Learning*, edited by Frank Hutter et al., Springer, 2019, pp. 3–33.
9. Gusită, Bianca, Cristina Guşiţă, and Daniela Stănescu. "A Machine Learning Framework for Stroke Prediction: Balancing Precision and Recall in Healthcare Analytics." *2025 IEEE Medical Measurements & Applications (MeMeA)*. IEEE, 2025.
10. Unal, Ilker. "Defining an optimal cut-point value in ROC analysis: an alternative approach." *Computational and mathematical methods in medicine* 2017.1 (2017): 3762651.
11. Di Cesare, Mariachiara et al. “The Heart of the World.” *Global heart* vol. 19,1 11. 25 Jan. 2024, doi:10.5334/gh.1288
12. MLFAMM. “Heart Disease Prediction: Comparing 6 ML Models.” *Kaggle*, 2023, https://www.kaggle.com/code/mlfamm/heart-disease-prediction-comparing-6-ml-models. Accessed 27 Apr. 2026.
13. Mohamed Elsadek. “Indicators of Heart Disease 2022 Update.” *Kaggle*, https://www.kaggle.com/code/mohamedelsadek44/indicators-of-heart-disease-2022-update. Accessed 27 Apr. 2026.
14. An, Qi, et al. "A comprehensive review on machine learning in healthcare industry: classification, restrictions, opportunities and challenges." *Sensors* 23.9 (2023): 4178.